Task 3 - Text Classification

Assignment 1

Maheen Nadeem –bscs23046

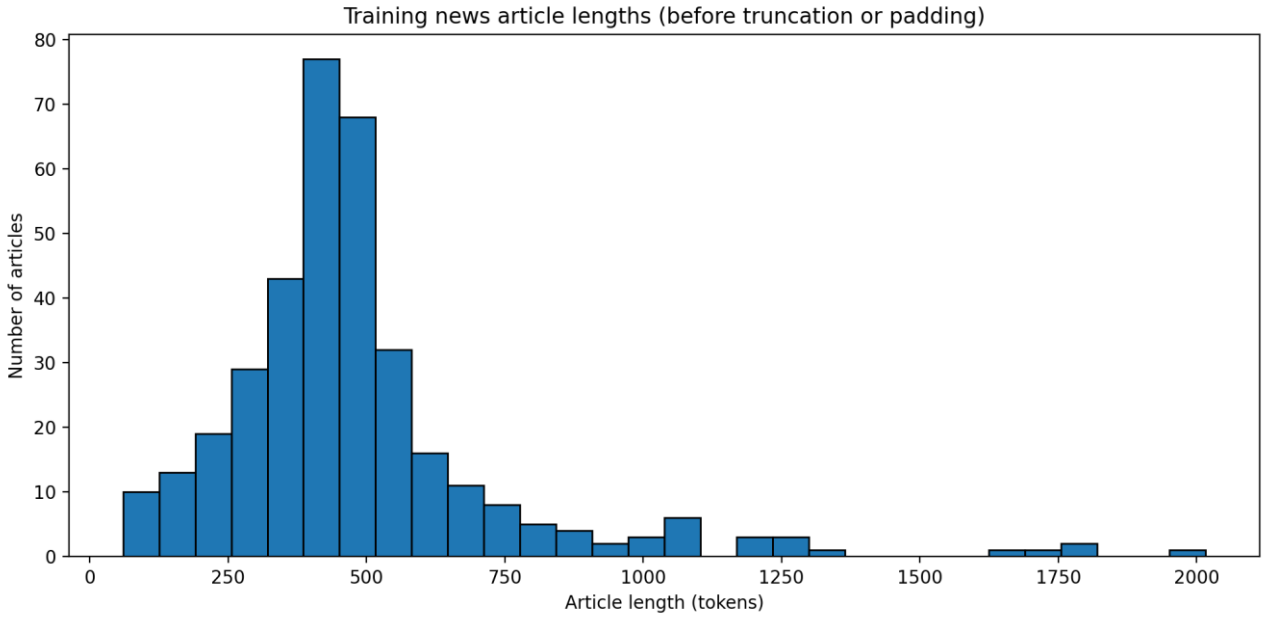
Experimental setup

This report evaluates binary classification of published and generated news articles using the supplied training, validation, and test splits. Each example is classified from article text only (real = 0, fake/generated = 1). The experiments use the starter EmbeddingBag model and an LSTM-based alternative. Unless stated otherwise, models are trained for five epochs with seed 42.

Dataset split: train 358 articles; validation 76; test 74.

3.1 Article-length distribution

Article lengths are counted as tokens from the original training articles, before any truncation or padding is applied. The histogram should show article length on the horizontal axis and article frequency on the vertical axis.



Most articles cluster around 300-550 tokens, with a long right tail. A 1,024-token cap retains the full length of most articles but truncates 19 long training examples. Padding is needed to combine variable-length articles into fixed-size batches; this changes computation, not the underlying labels

3.2 Truncation, padding and runtime

TruncateToMaxLen retains the first max_len tokens in each article. PadToMaxLen then appends the custom padding index to sequences shorter than that limit. Setting max_len = -1 disables both transformations.

3.2(a) Runtime comparison

All three runs use CPU, five epochs, and the same training/validation/test splits. Enter the training-and-evaluation runtime printed at the end of each run.

Batch size	Maximum length	Runtime (seconds)
1	None (-1)	*7.390*
1	1,024	*7.007*
32	1,024	*4.340*

With max_len = 1,024, increasing batch size from 1 to 32 reduced runtime from 7.007 s to 4.340 s, a 38.1% reduction. Batching reduces optimizer steps and Python overhead. The 0.383 s difference between the two batch-size-1 runs is small and may partly reflect measurement noise.

3.2(b) Error with batch size 32 and no padding

Command: python txt_classifier.py --batch_size 32 --max_len -1

RuntimeError: stack expects each tensor to be equal size, but got [140] at entry 0 and [517] at entry 1

3.2(c) Explanation of the error

When max_len = -1, article sequences are neither truncated nor padded, so their token counts differ. The PadSequence collator uses torch.stack(), which requires equal-sized tensors. A batch of 32 therefore fails when two articles have different lengths. Batch size 1 succeeds because each batch contains only one sequence.

3.3 Comparing SGD and Adam

The original EmbeddingBag classifier is trained with the assignment-specified optimizer configurations: SGD at learning rate 5 and Adam at learning rate 0.001. All other experiment settings are kept consistent. Enter the final epoch validation accuracy and the separately reported test accuracy.

Optimizer (learning rate)	Train accuracy	Validation accuracy	Test accuracy
SGD (5)	77.65	85.53	83.78
Adam (0.001)	94.41	85.53	91.89

Both optimizers reached 85.53% validation accuracy. Adam reached 91.89% test accuracy compared with SGD's 83.78% (+8.11 percentage points), with more consistent growth in training accuracy. Test loss was lower for SGD (0.4255 versus 0.5730 for Adam), so accuracy and loss do not rank the optimizers identically. Because the learning rate also changed, this compares the requested optimizer configurations rather than isolating the optimizer alone.

3.4 LSTM text classifier

3.4(a) Architecture / computation graph

The alternative classifier converts token IDs to 64-dimensional embeddings using nn.Embedding, processes sequences using a single-layer unidirectional LSTM with 64 hidden units and batch_first=True, masks padding positions, and applies adaptive max pooling over the valid sequence positions. Dropout (0.2) and a linear layer generate logits for two classes.

Token IDs [B, 1024]	Embedding [B, 1024, 64]	LSTM [B, 1024, 64]	Masked max [B, 64]	Dropout p = 0.2	Linear [B, 2]

LSTM classifier computation stages (B denotes batch size).

The input and output shapes shown above assume max_len = 1,024, embed_dim = 64 and hidden_dim = 64.

3.4(b) Accuracy comparison

Both models are evaluated on the supplied splits after training with Adam (learning rate 0.001), batch size 32, maximum sequence length 1,024 and five epochs.

Model	Final train accuracy	Validation accuracy	Test accuracy
Original EmbeddingBag + Linear	94.41	85.53	91.89
Embedding + LSTM + Pooling	97.77	89.47	91.89

3.4(c) Interpretation

The LSTM improved validation accuracy by 3.94 percentage points and training accuracy by 3.36 points; both models achieved 91.89% test accuracy. Learning from token order may help the LSTM capture phrases and local context lost by averaged embeddings. The larger train-validation gap (97.77% versus 89.47%) also suggests that generalization could be limited on this small dataset. Final validation and test losses were higher for LSTM (0.6199 / 0.6264) than the baseline (0.5821 / 0.5730), so the improvement is not uniform across metrics. Padding was excluded from pooling, although the LSTM still processes the padded positions.

3.5 Observations from real/fake article pairs

The provided news dataset contains paired articles that share a headline: a published article and an LLM-generated article based on that headline. Two inspected pairs illustrate stylistic differences that the classifier may exploit.

Article pair	Published article	Generated article
Panda Le Le flight	Contains concrete travel details, including a departure date and airport names.	Uses elaborate descriptions such as a specially designed bamboo-themed first-class cabin.
Buffalo Grove scholarship	Identifies the student, school, scholarship organization and sponsoring club.	Describes an unnamed student and emphasizes general praise and inspirational wording.

These examples suggest that the model may learn differences in specificity, named entities, vocabulary and tone. Published articles often contain precise facts or attributed details, while generated articles can be more general or embellished. This does not make the classifier a general-purpose fact checker: it predicts published/generated source labels within the supplied dataset.

Experiment settings and limitations

Setting	Configuration
Data splits	358 training / 76 validation / 74 test articles
Baseline	EmbeddingBag (mean pooling) + Linear
LSTM	Embedding 64; LSTM hidden 64; masked max pooling; dropout 0.2
Optimization	SGD, learning rate 5; Adam, learning rate 0.001
Shared settings	Batch size 32; max_len 1,024; five epochs; seed 42
Device	Runtime / baseline: CPU; LSTM: CUDA

These findings represent a single seed and a single train/validation/test split rather than repeated-run estimates. CPU and GPU runtimes should not be directly compared. Final reported accuracies should be interpreted with these limitations in mind.

Saved model checkpoints

Saved trained-model checkpoints are available in the following Google Drive folder.

Google Drive: https://drive.google.com/drive/folders/1Az7Jv03aZqb2gsqUduyvbe676Sk-cfyq?usp=sharing